

Как устроена авторизация, двухфакторка и смена пароля

Пошаговый визуальный разбор: что видит пользователь на экране, какой именно запрос в этот момент улетает на сервер (реальная панель Network из Chrome DevTools, крупно — с содержимым запроса, то есть Payload) и что сервер отвечает, и что происходит внутри бэкенда — какие проверки, какой код и какая логика отработывают за эти доли секунды. Документ рассчитан на читателя без опыта в разработке.

-  Скриншот приложения — то, что видит пользователь в браузере
-  Панель Network из Chrome DevTools: Payload — что отправлено, Preview — что получено, Headers — адрес и заголовки
-  Тёмные блоки — реальный код бэкенда, с подсветкой

01 Вход в аккаунт, когда включена 2FA

02 Отключение двухфакторной аутентификации

03 Обычный вход (2FA выключена)

04 Повторное включение 2FA

05 Смена пароля

06 Редактирование профиля

07 Что защищает аккаунт: итоги

1 Вход в аккаунт, когда включена 2FA

У аккаунта сейчас включена двухфакторная аутентификация (2FA). Это значит, что одного пароля недостаточно — после него сервер потребует ещё одноразовый код из письма. Смотрим, как это выглядит и что происходит на сервере на каждом шаге.

Как читать скриншоты панели Network

Под каждым действием, которое обращается к серверу, показана настоящая панель Network из Chrome DevTools. Сверху — метод и адрес запроса и код ответа (200 = «всё хорошо»). Ниже по вкладкам: **Payload** — данные, которые браузер отправил серверу (то, что вы «положили в конверт»); **Preview** — данные, которые сервер прислал в ответ; **Headers** — служебные «пометки на конверте» (адрес, метод, заголовок Authorization с токеном). Если запрос ничего не отправляет в теле (например, просто «дай мой профиль»), вкладки Payload у него нет — тогда показан Headers.

Шаг 1 — заполняем форму входа FRONTEND

Sign In

Access the platform admin console

Work Email

Password [Forgot Password?](#)

[Sign in](#)

Форма входа заполнена, кнопка «Sign in» ещё не нажата. В Network при этом пусто — ничего не отправлено.

Пока вы печатаете email и пароль, в сеть ничего не уходит — проверяется только на глаз, что поля не пустые. Реальная проверка происходит только после нажатия «Sign in».

← Back

Enter 2FA otp

We sent a 6 digit otp to shipinomaksim@gmail.com.

Verify 2FA OTP

Браузер сам переключился на страницу ввода кода: сервер ответил, что для входа нужен второй шаг.

POST

http://localhost:4000/auth/login

200 OK

→ Payload — что браузер отправил на сервер (тело запроса)

Elements

Console

Sources

Network

Performance

Memory

Application

Security

>>

Name

X

Headers

Payload

Preview

Response

Initiator

Timing

login

Request payload

View source

{email: "shipinomaksim@gmail.com", password: "33333333"}
email: "shipinomaksim@gmail.com"
password: "33333333"

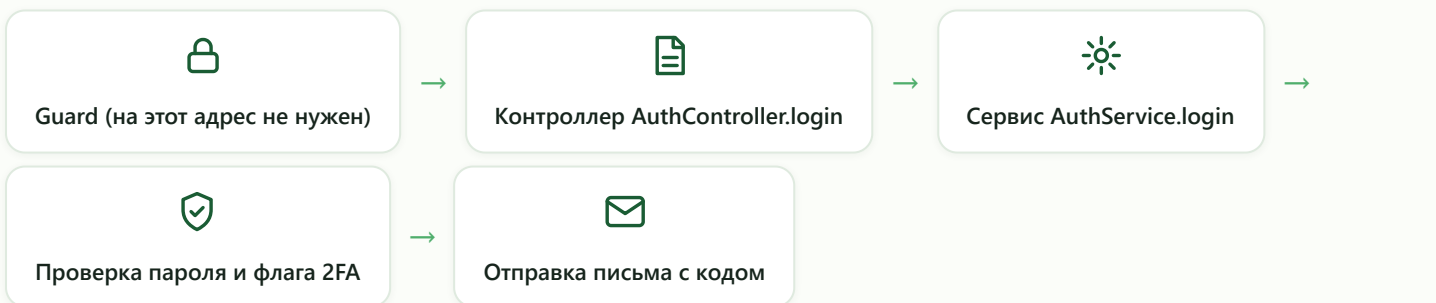
A screenshot of the Chrome DevTools Network tab. The 'Network' tab is selected, and the 'Preview' sub-tab is active. The request name is 'login'. The response body is expanded, showing a JSON object:

```
{twoFactorRequired: true,...}
  message: "Otp send successfully"
  twoFactorRequired: true
  twoFactorToken: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySW0..."
```

Что улетело на сервер и что вернулось

Payload — всего два поля: `email` и `password`. Это тело запроса: маленький JSON-документ, который браузер положил в запрос. Пароль виден открытым текстом — внутри запроса так и должно быть: в пути его защищает HTTPS-соединение (в боевой среде оно обязательно; здесь, на `localhost`, шифрования нет и это нормально для разработки), а на сервере он сразу превращается в хэш для сравнения и нигде не сохраняется. **Preview** — в ответе **нет** ключей доступа: только `twoFactorRequired: true`, `message` и `twoFactorToken` — длинная подписанная строка, «билет на второй шаг».

Это ключевой момент: запрос `login` вернул **200 OK** (пароль верный!), но без токенов доступа. Вместо них сервер прислал короткоживущий «билет на второй шаг». Разберём, что произошло внутри:



Простыми словами. Вход не защищён "охранником" (Guard'ом), потому что на этом шаге пользователь ещё не залогинен — охранять нечего, наоборот, сюда должен суметь достучаться кто угодно с логином и паролем. А вот дальше контроллер (это как диспетчер: принял запрос, передал по адресу и вернул ответ) отдаёт всю работу сервису. Сервис:

- 1 Находит пользователя по email в базе данных.
- 2 Сверяет пароль с тем, что хранится в базе (в базе лежит не сам пароль, а его необратимый «отпечаток» — хэш; сравнение идёт по отпечаткам).
- 3 Видит, что у аккаунта включена 2FA — и вместо выдачи токенов генерирует 6-значный код, сохраняет его (тоже в виде хэша, не в открытом виде!) и отправляет на почту через сервис Resend.

- 4 Подписывает специальный короткоживущий токен (живёт 15 минут) — это тот самый «билет на второй шаг», который видно в ответе на скриншоте как `twoFactorToken`.

```
// backend/src/auth/auth.service.ts
async login(email: string, password: string, context = {}) {
  const user = await this.usersService.findByEmail(email);

  // Сравнение с "фиктивным" хэшем, даже если пользователь не найден -
  // чтобы ответ занимал одинаковое время в обоих случаях (защита от
  // подбора существующих email по разнице в скорости ответа).
  const passwordMatches = await bcrypt.compare(
    password,
    user?.passwordHash ?? ABSENT_USER_PASSWORD_HASH,
  );

  if (!user || !passwordMatches) {
    throw new UnauthorizedException('Invalid email or password');
  }

  if (user.twoFactorEnabled) {
    // Не выдаём токены сразу - создаём одноразовый код и билет
    // на второй шаг проверки.
    const result = await this.twoFactorCreateOtpToken(user.id);
    return {
      twoFactorRequired: true,
      twoFactorToken: result.twoFactorToken,
      message: result.message,
    };
  }

  const tokens = await this.issueTokenPair(user, context);
  return { ...tokens, userType: user.userType };
}
```

Почему пароль сравнивают даже для несуществующего email?

Это защита от «угадывания» зарегистрированных адресов. Если бы для несуществующего email сервер отвечал мгновенно, а для существующего - с небольшой задержкой (на сравнение пароля), по одной только скорости ответа можно было бы понять, зарегистрирован ли email в системе. Поэтому сравнение хэшей выполняется всегда, даже вхолостую.

Шаг 3 — вводим код из письма FRONTEND

[← Back](#)

Enter 2FA otp

We sent a 6 digit otp to shipinomaksim@gmail.com.

5

5

0

3

1

3

Verify 2FA OTP

Шестизначный код, только что пришедший на почту, введён в поле.

Шаг 4 — код проверен, доступ открыт FRONTEND BACKEND

Platform

Dashboard

Sellers

Commision & Payouts

Global Catalog

Delivery Network

Customers

Orders

Supports

Marketing

Reports

Users & Permissions

Settings

Shipino Maksim
shipinomaksim@gmail.com

Q search store, order, agent...

DashboardPage

После кнопки «Verify»: вход выполнен, открылся дашборд.

200 OK

The screenshot shows the Chrome DevTools Network tab. The 'Payload' sub-tab is selected, displaying the request body. The request is identified as '2fa-verify-login-otp'. The payload is a JSON object containing an 'otp' field with the value '550313' and a 'twoFactorToken' field with a long, complex alphanumeric string. A 'View source' button is located next to the 'Request payload' header.

[illegible]

Payload — два поля: `otp` (шесть цифр из письма) и `twoFactorToken` (тот самый «билет» из предыдущего ответа). Сервер ничего не помнит между запросами, поэтому браузер сам возвращает билет вместе с кодом. **Preview** — теперь в ответе настоящие ключи: `accessToken`, `refreshToken` и `userType`.

GET

http://localhost:4000/auth/me

200 OK

→ Headers — адрес, метод и заголовки запроса (в том числе Authorization)

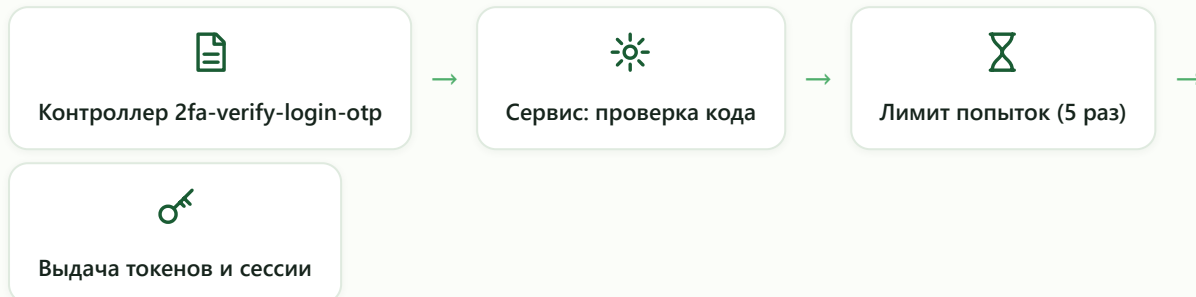
Name	Headers	Preview	Response	Initiator	Timing
2fa-verify-login-otp	▼ General				
me	Request URL		http://localhost:4000/auth/me		
me	Request Method		GET		
	Status Code		200 OK		
	Remote Address		:::1:4000		
	Referrer Policy		strict-origin-when-cross-origin		
	▶ Response headers (24)				
	▼ Request headers	☐ Raw			
	Accept		application/json, text/plain, */*		
	Accept-Encoding		gzip, deflate, br, zstd		
	Accept-Language		uk-UA,uk;q=0.9,en-US;q=0.8,en;q=0.7		
	Authorization		Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJjbXU1cXZtNGUwMDAwamN2Nm04NDI6ZTJpIiwiaWF0IjZaGlwaW5vbWFRc2ltQGdtYWlsLmNvbSIsInVzZXJ1eXB1IjoieUEExBVEZPuk1fQURN SU4iLCJzaWQiOiJnNGFoMXhzbGw5YmFoMXIxZHI1NTY2bzUiLCJpYXQ iOjE3ODk4MzExNDIsImV4cCI6MTc4OTgzMjA0Mn0.18FjwANQcEqzUI NMGINGhEJwzM9v7SrNiGPefRxosa0		
	Connection		keep-alive		

← Preview — что сервер ответил

Name	Headers	Preview	Response	Initiator	Timing
2fa-verify-login-otp		▼	{id: "cmu5qvm4e0000jcv6j849ze2i", email: "shipinomaksim@gmail.com"		
me			createdAt: "2026-09-17T16:29:16.096Z"		
me			email: "shipinomaksim@gmail.com"		
			fullName: "Shipino Maksim"		
			id: "cmu5qvm4e0000jcv6j849ze2i"		
			phone: "3850006879"		
			profileImage: "https://github.com/shadcn.png"		
			status: "ACTIVE"		
			twoFactorEnabled: true		
			updatedAt: "2026-09-18T13:36:46.770Z"		
			userType: "PLATFORM_ADMIN"		

Как сервер узнаёт, кто спрашивает

У этого запроса нет тела — браузер просто просит «дай мой профиль», поэтому вкладка **Payload** отсутствует. Зато в **Headers** видно главное: заголовок `Authorization: Bearer <accessToken>` — тот самый ключ доступа, выданный секундой раньше. В **Preview** приходят данные профиля (имя, email, телефон, тип пользователя и т.д.) — без пароля и секретов 2FA.



Простыми словами. Сервер достаёт из базы последний код, который предназначен именно «для входа» (а не, например, для включения 2FA — у каждого кода есть метка «назначения», чтобы код от одного действия нельзя было подсунуть в другое). Сверяет введённое с хэшем. Если код неверный — увеличивает счётчик неудачных попыток; после пятой неудачной попытки код перестаёт приниматься вообще, даже если ввести его правильно — это защита от перебора всех 900 000 возможных комбинаций.

```
// backend/src/auth/auth.service.ts

async twoFactorVerifyLoginOtp(twoFactorToken: string, otp: string, context = {}) {
  // 1. Билет на второй шаг должен быть подлинным и ещё не истёкшим.
  let decode: TwoFactorTokenPayload;
  try {
    decode = await this.jwtService.verifyAsync(twoFactorToken, {
      secret: this.config.getOrThrow<string>('JWT_2FA_SECRET'),
    });
  } catch {
    throw new BadRequestException('Session is Expired');
  }

  const user = await this.prisma.user.findUnique({ where: { id: decode.userId } });
  if (user.status !== UserStatus.ACTIVE) {
    throw new ForbiddenException('This account is not active');
  }

  // 2. Проверка самого кода: назначение "LOGIN", лимит попыток, хэш.
  await this.verifyOtp(user.id, TwoFactorOtpPurpose.LOGIN, otp);

  // 3. Всё сошлось - только теперь выдаём настоящие токены доступа.
  const token = await this.issueTokenPair(user, context);
  return { ...token, userType: user.userType };
}
```

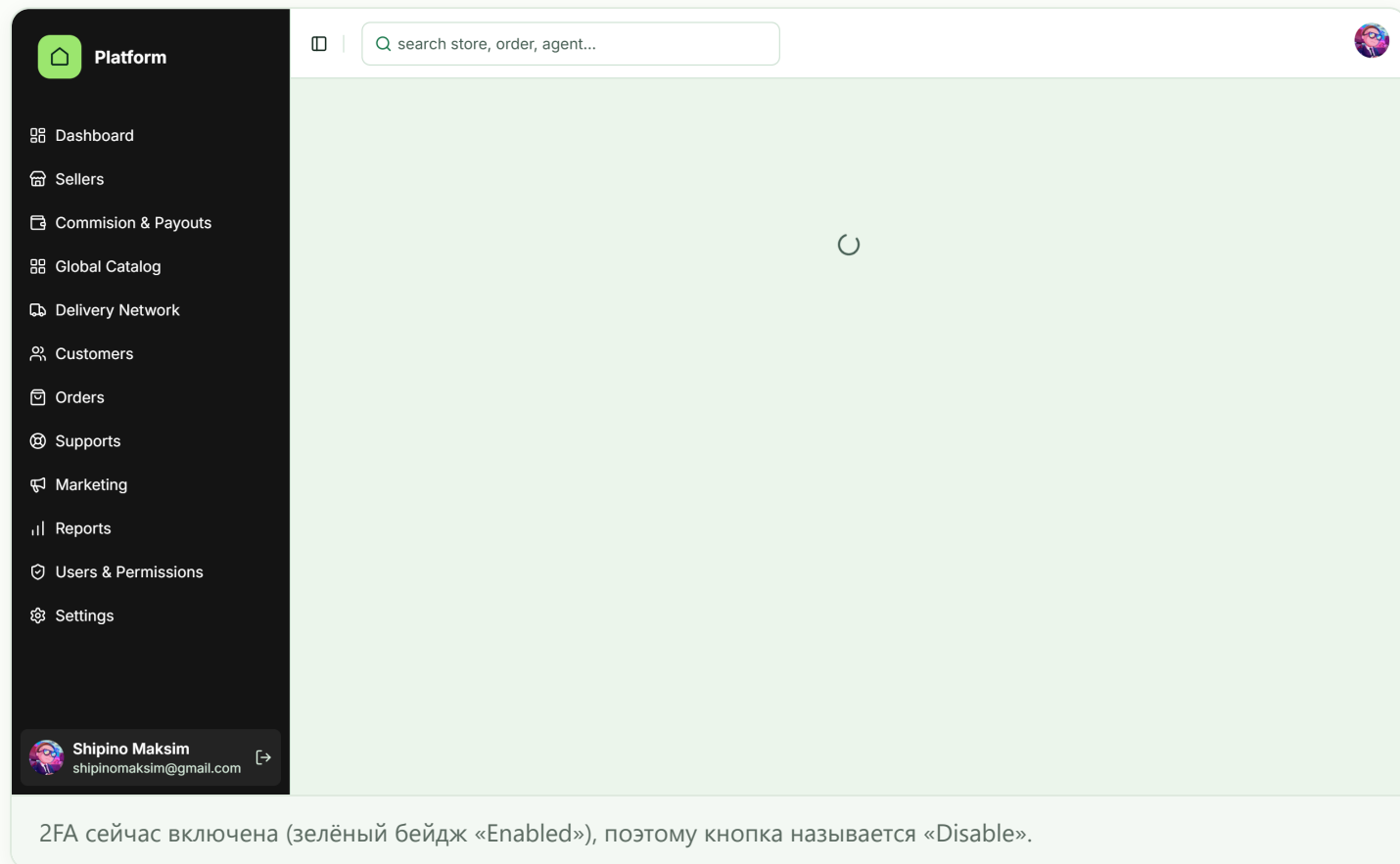
А что за два запроса "me" подряд?

Первый - GET /auth/me сразу после входа: страница дашборда запрашивает профиль, чтобы показать имя и аватар. Второй запрос с кодом 304 - это React в строгом режиме разработки (Strict Mode) намеренно повторяет эффект, чтобы разработчики ловили побочные эффекты; сервер отвечает "содержимое не изменилось", лишней нагрузки это не создаёт.

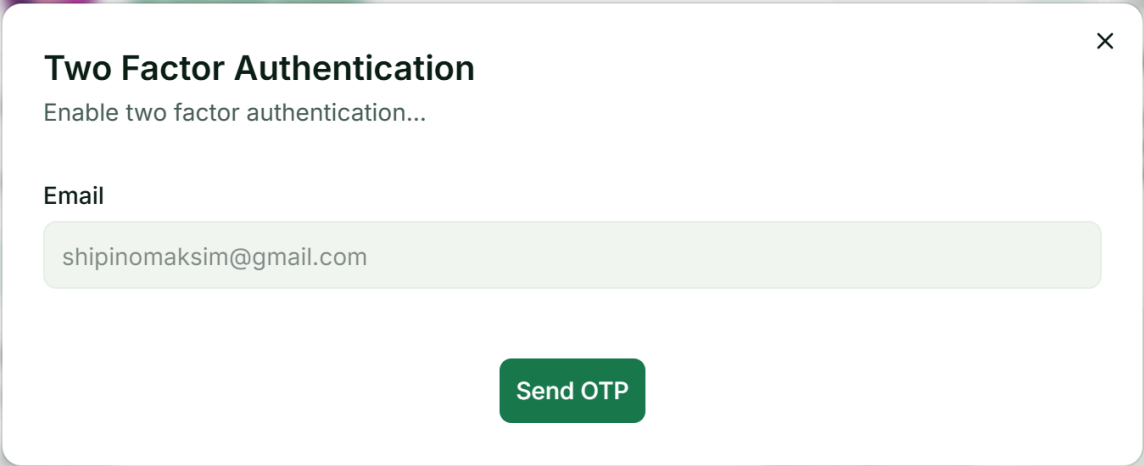
2 Отключение двухфакторной аутентификации

Раздел «Мой аккаунт» → «Password & Security». Включение и выключение 2FA технически — один и тот же переключатель на сервере: он всегда ставит значение, противоположное текущему. Поэтому что «включить», что «выключить» проходит через один и тот же код, только с разным начальным состоянием.

Шаг 1 — страница аккаунта **FRONTEND**



Шаг 2 — открываем диалог **FRONTEND**



Two Factor Authentication ×

Enable two factor authentication...

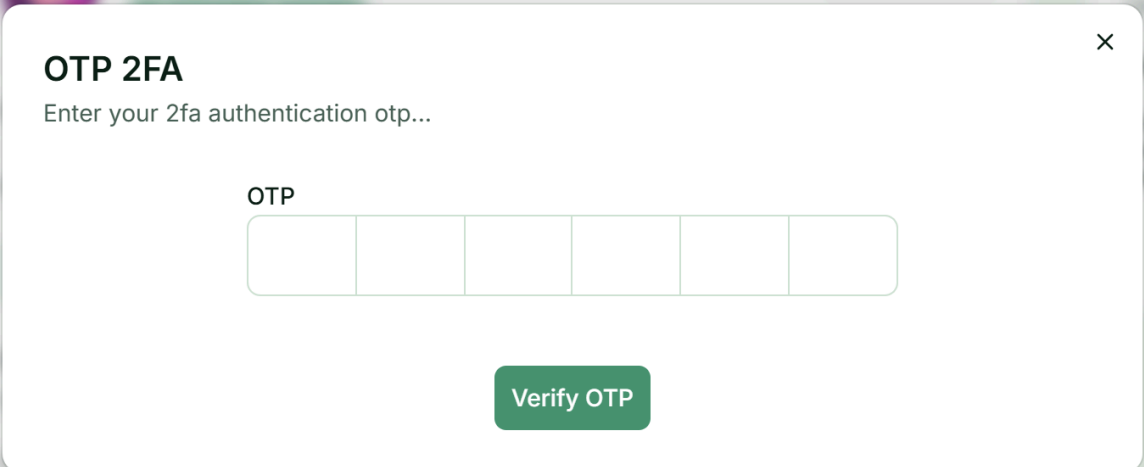
Email

shipinomaksim@gmail.com

Send OTP

Диалог из двух шагов: сначала подтверждаем e-mail и просим код.

Шаг 3 — код отправлен **FRONTEND** **BACKEND**



OTP 2FA ×

Enter your 2fa authentication otp...

OTP

Verify OTP

Код создан и отправлен на почту; форма переключилась на ввод шести цифр.

POST

http://localhost:4000/auth/2fa-generate-otp

200 OK

→ Headers — адрес, метод и заголовки запроса (в том числе Authorization)

Elements

Console

Sources

Network

Performance

Memory

Application

Security

>>

5

⚙

⋮

Name

✕

Headers

Preview

Response

Initiator

Timing

2fa-generate-otp

▼ General

Request URL

http://localhost:4000/auth/2fa-generate-otp

Request Method

POST

Status Code

200 OK

Remote Address

[::1]:4000

Referrer Policy

strict-origin-when-cross-origin

▼ Response headers

☐ Raw

Access-Control-Allow-Credentials

true

Access-Control-Allow-Origin

http://localhost:3002

Connection

keep-alive

Content-Length

50

Content-Security-Policy

default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action 'self';frame-ancestors 'self';img-src 'self' data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' https: 'unsafe-inline';upgrade-insecure-requests

Content-Type

application/json; charset=utf-8

← Preview — что сервер ответил

Elements

Console

Sources

Network

Performance

Memory

Application

Security

>>

Name

✕

Headers

Preview

Response

Initiator

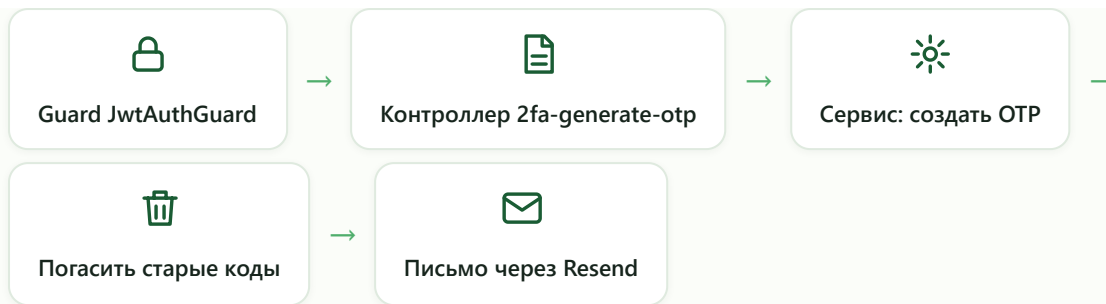
Timing

2fa-generate-otp

▼ {success: true, message: "Otp send successfully"}
message: "Otp send successfully"
success: true

Что улетело на сервер и что вернулось

Тела у запроса нет (нечего передавать — сервер и так знает, кто вы по заголовку Authorization), хотя метод POST: он здесь просто означает «выполни действие», поэтому показан **Headers**: адрес, метод POST и заголовок Authorization: Bearer В **Preview** — только success: true и сообщение Otp send successfully . Обратите внимание: самого кода в ответе **нет** — он уходит исключительно на почту, иначе любой, кто перехватит ответ, получил бы и код.



Простыми словами про Guard. В отличие от входа, этот запрос требует, чтобы человек уже был залогинен — поэтому перед контроллером стоит «охранник» `JwtAuthGuard`. Он смотрит на заголовок `Authorization: Bearer ...`, присланный браузером, проверяет подпись токена и, если всё в порядке, сверяется с базой — жива ли ещё сессия (её могли принудительно отозвать, например, при выходе с другого устройства). Если что-то не так — запрос отклоняется ошибкой 401 ещё до того, как выполнится хоть строчка бизнес-логики.

```
// backend/src/common/guards/jwt-auth.guard.ts
@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {}

// backend/src/auth/strategies/jwt.strategy.ts
async validate(payload: JwtAccessPayload): Promise<AuthenticatedUser> {
  const session = await this.prisma.userSession.findUnique({
    where: { id: payload.sid },
    include: { user: true },
  });

  // Сессию могли отозвать вручную (например, кнопкой "выйти со всех устройств") - тогда токен всё ещё "живой" по подписи, но сервер всё равно откажет.
  if (!session || session.revokedAt || session.expiresAt <= new Date()) {
    throw new UnauthorizedException('Session is no longer valid');
  }
  if (session.user.status !== UserStatus.ACTIVE) {
    throw new UnauthorizedException('This account is not active');
  }

  return { userId: session.userId, email: session.user.email,
    userType: session.user.userType, sessionId: session.id };
}
```

Шаг 4 — вводим код

FRONTEND

OTP 2FA

Enter your 2fa authentication otp...

OTP

9

3

9

9

7

0

Verify OTP

Код из письма введён, готовы нажать «Verify OTP».

Шаг 5 — 2FA выключена

FRONTEND

BACKEND

Platform

Dashboard

Sellers

Commision & Payouts

Global Catalog

Delivery Network

Customers

Orders

Supports

Marketing

Reports

Users & Permissions

Settings

Shipino Maksim

shipinomaksim@gmail.com

search store, order, agent...

My account

Your profile, password, two-factor authentication, and where you're signed in

Shipino Maksim

PLATFORM_ADMIN

Edit profile

Email

shipinomaksim@gmail.com

Phone

3850006879

Role & Access

Update your profile details

Role

-

Full access to every module -stores, catalog, payouts, delivery, network, and platform settings.

View permission matrix →

Password & Security

Manage your password and two-factor authentication

Password

Last changed 2 months ago

Change Password

Two-factor authentication

Extra code required at login, in addition to your password

Disabled

Enable

Active sessions

Devices where you're currently signed in

3 devices signed in

Two-factor disabled

На карточке появляется актуальное состояние переключателя — 2FA выключена.

POST

http://localhost:4000/auth/2fa-verify-enable

200 OK

→ Payload — что браузер отправил на сервер (тело запроса)

ElementsConsoleSourcesNetworkPerformanceMemoryApplicationSecurity>>

NameXHeadersPayloadPreviewResponseInitiatorTiming

2fa-verify-enable

Request payloadView source

{otp: "939970"}
otp: "939970"

← Preview — что сервер ответил

ElementsConsoleSourcesNetworkPerformanceMemoryApplicationSecurity>>

NameXHeadersPayloadPreviewResponseInitiatorTiming

2fa-verify-enable

{success: true, message: "Two-factor disabled", data: {twoFactorEnabled: false}}

data: {twoFactorEnabled: false}
message: "Two-factor disabled"
success: true

Что улетело на сервер и что вернулось

Payload — одно поле: `otp`. В запросе нет ни слова «выключить»: сервер сам решает, что делать, и просто переключает значение на противоположное. **Preview**: `message: "Two-factor disabled"` и `data.twoFactorEnabled: false` — итоговое состояние, которое сервер записал в базу.

```
// backend/src/auth/auth.service.ts

async twoFactorVerifyEnable(userId: string, otp: string) {
  const user = await this.prisma.user.findUnique({ where: { id: userId } });

  // Код обязательно должен быть с меткой ENABLE_TOGGLE - код, присланный
  // для входа, здесь не примут, даже если он ещё действителен.
  await this.verifyOtp(userId, TwoFactorOtpPurpose.ENABLE_TOGGLE, otp);

  const updatedUser = await this.prisma.user.update({
    where: { id: user.id },
    data: { twoFactorEnabled: !user.twoFactorEnabled }, // переключатель
  });

  return {
    success: true,
    message: `Two-factor ${updatedUser.twoFactorEnabled ? 'enabled' : 'disabled'}`,
    data: { twoFactorEnabled: updatedUser.twoFactorEnabled },
  };
}
```

Зачем коду "метка назначения"?

Коды для входа и коды для включения/выключения 2FA генерируются одной и той же функцией и хранятся в одной таблице. Без метки назначения (purpose) код, присланный для входа, можно было бы подсунуть в форму отключения 2FA - и наоборот. Метка гарантирует, что код, полученный для одной цели, не сработает для другой.

3 Обычный вход (2FA выключена)

Сейчас 2FA временно выключена (предыдущий раздел). Для сравнения — вот как выглядит вход без второго шага: пароль подошёл — и сразу выданы токены доступа.

Шаг 1 — выходим и заходим заново FRONTEND

Sign In

Access the platform admin console

Work Email

you@platform.com

Password

.....

Forgot Password?

Sign in

Кнопка «Log out» в боковом меню — сессия на сервере отозвана, локальные токены стёрты. Вернулись на страницу входа.

Снова вводим email и пароль — те же, что и раньше.

Platform

Dashboard

Sellers

Commision & Payouts

Global Catalog

Delivery Network

Customers

Orders

Supports

Marketing

Reports

Users & Permissions

Settings

Shipino Maksim
shipinomaksim@gmail.com

search store, order, agent...

DashboardPage

Пароль подошёл — и сразу дашборд, без промежуточного шага с кодом.

POST

http://localhost:4000/auth/login

200 OK

→ Payload — что браузер отправил на сервер (тело запроса)

Elements

Console

Sources

Network

Performance

Memory

Application

Security

>>

Name

login

me

me

Headers

Payload

Preview

Response

Initiator

Timing

Request payload

View source

{email: "shipinomaksim@gmail.com", password: "33333333"}
email: "shipinomaksim@gmail.com"
password: "33333333"

← Preview — что сервер ответил

A screenshot of the Chrome DevTools Network tab. The left sidebar shows a list of network requests: "login", "me", and another "me". The "login" request is selected. The main panel displays the response preview for the "login" request, which is a JSON object containing three fields: "accessToken", "refreshToken", and "userType".

Сравните с входом при включённой 2FA

Payload тот же самый: `email` и `password`. Разница — в **Preview**: вместо «билета на второй шаг» сервер сразу присылает `accessToken`, `refreshToken` и `userType`.

Это тот же самый метод `AuthService.login`, что и в разделе 1, просто условие `if (user.twoFactorEnabled)` в этот раз не срабатывает — и код сразу переходит к выдаче токенов:

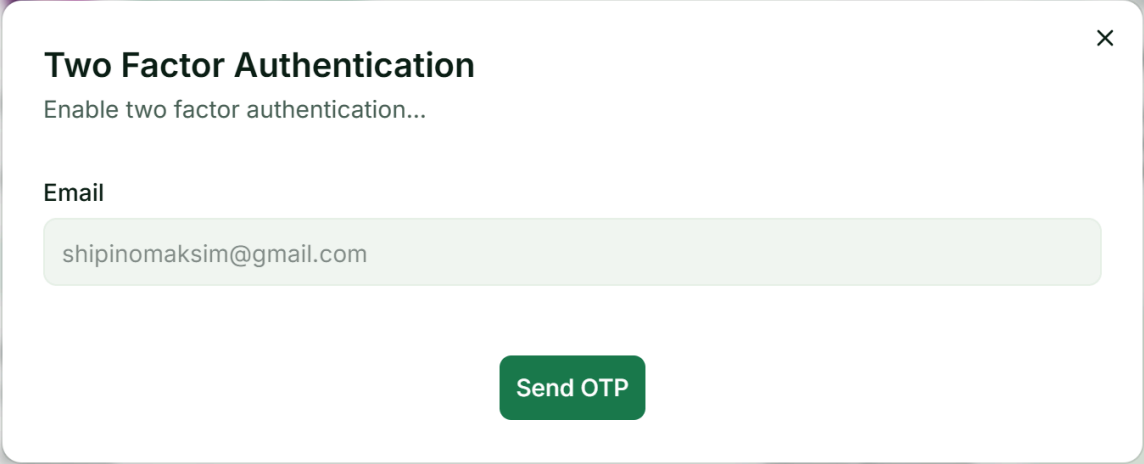
```
const tokens = await this.issueTokenPair(user, context);
return { ...tokens, userType: user.userType };
```

Что именно выдаётся

Два токена: короткоживущий access-токен (по нему сервер узнаёт "кто вы" при каждом запросе, живёт 15 минут) и долгоживущий refresh-токен (нужен только для того, чтобы незаметно получить новый access-токен, когда старый истечёт). Плюс в базе создаётся запись о сессии - устройство, IP, время входа - её видно в разделе «Active sessions».

4 Повторное включение 2FA

Возвращаем защиту обратно — ровно тот же переключатель, что и в разделе 2, только в другую сторону.



Two Factor Authentication ×

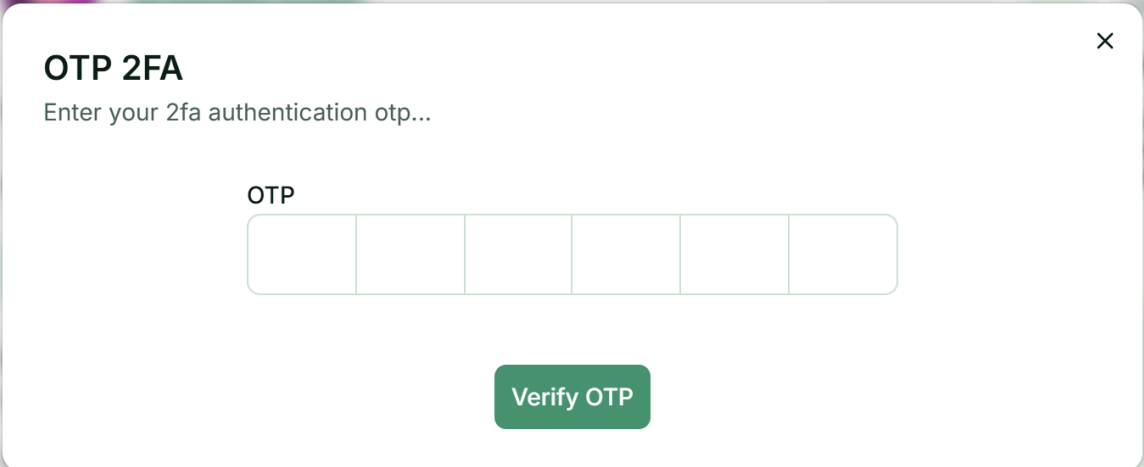
Enable two factor authentication...

Email

shipinomaksim@gmail.com

Send OTP

Кнопка снова называется «Enable» — 2FA сейчас выключена.



OTP 2FA ×

Enter your 2fa authentication otp...

OTP

Verify OTP

Новый код сгенерирован и отправлен на почту.

POST

http://localhost:4000/auth/2fa-generate-otp

200 OK

← Preview — что сервер ответил

Elements

Console

Sources

Network

Performance

Memory

Application

Security

>>

Name

2fa-generate-otp

×

Headers

Preview

Response

Initiator

Timing

▼ {success: true, message: "Otp send successfully"}
message: "Otp send successfully"
success: true

OTP 2FA

×

Enter your 2fa authentication otp...

OTP

7

2

1

1

4

5

Verify OTP

Код введён.

Platform

Dashboard

Sellers

Commision & Payouts

Global Catalog

Delivery Network

Customers

Orders

Supports

Marketing

Reports

Users & Permissions

Settings

Shipino Maksim

shipinomaksim@gmail.com

Q search store, order, agent...

My account

Your profile, password, two-factor authentication, and where you're signed in

Shipino Maksim

PLATFORM_ADMIN

Edit profile

Email

shipinomaksim@gmail.com

Phone

3850006879

Role & Access

Update your profile details

Role

-

Full access to every module -stores, catalog, payouts, delivery, network, and platform settings.

View permission matrix →

Password & Security

Manage your password and two-factor authentication

Password

Last changed 2 months ago

Change Password

Two-factor authentication

Extra code required at login, in addition to your password

Enabled

Disable

Active sessions

Devices where you're currently signed in

3 devices signed in

Two-factor enabled

2FA снова включена.

POSThttp://localhost:4000/auth/2fa-verify-enable200 OK

→ Payload — что браузер отправил на сервер (тело запроса)

ElementsConsoleSourcesNetworkPerformanceMemoryApplicationSecurity>>

Name

2fa-verify-enable

Request payload

{otp: "721145"}

otp: "721145"

← Preview — что сервер ответил

⌵		Elements	Console	Sources	Network	Performance	Memory	Application	Security	>>
Name	✕	Headers	Payload	Preview	Response	Initiator	Timing			
2fa-verify-enable	▼	{success: true, message: "Two-factor enabled", data: {twoFactorEnabled: true}}								
	▶	data: {twoFactorEnabled: true}								
		message: "Two-factor enabled"								
		success: true								

Тот же запрос — противоположный результат

В **Payload** снова только `otp`, ровно как при выключении. Отличается лишь **Preview**: теперь сервер вернул `twoFactorEnabled: true`.

Итог

Код внутри - буквально тот же самый метод `twoFactorVerifyEnable` из раздела 2. Он не спрашивает "включить" или "выключить" - он просто ставит противоположное значение тому, что было. Именно поэтому фронтенд сам решает, какую надпись показать на кнопке ("Enable" или "Disable"), глядя на текущее состояние.

5 Смена пароля

Отдельная форма, отдельный адрес на сервере, ничего общего с редактированием профиля — и это осознанное решение, разберём почему.

Шаг 1 — заполняем форму **FRONTEND**

×

Change Password

Update your password here...

Current Password

.....

👁

Password

.....

👁

Confirm Password

.....

👁

Save changes

Текущий пароль, новый пароль и его подтверждение.

Platform

Dashboard

Sellers

Commision & Payouts

Global Catalog

Delivery Network

Customers

Orders

Supports

Marketing

Reports

Users & Permissions

Settings

Shipino Maksim

shipinomaksim@gmail.com

search store, order, agent...

My account

Your profile, password, two-factor authentication, and where you're signed in

Shipino Maksim

PLATFORM_ADMIN

Edit profile

Email

shipinomaksim@gmail.com

Phone

3850006879

Role & Access

Update your profile details

Role

-

Full access to every module -stores, catalog, payouts, delivery, network, and platform settings.

View permission matrix →

Password & Security

Manage your password and two-factor authentication

Password

Last changed 2 months ago

Change Password

Two-factor authentication

Extra code required at login, in addition to your password

Enabled

Disable

Active sessions

3 devices signed in

Devices where you're currently signed in

Password changed successfully

Уведомление «Password changed successfully».

PUThttp://localhost:4000/users/change-password200 OK

→ Payload — что браузер отправил на сервер (тело запроса)

ElementsConsoleSourcesNetworkPerformanceMemoryApplicationSecurity

Name

change-password

HeadersPayloadPreviewResponseInitiatorTiming

Request payload

View source

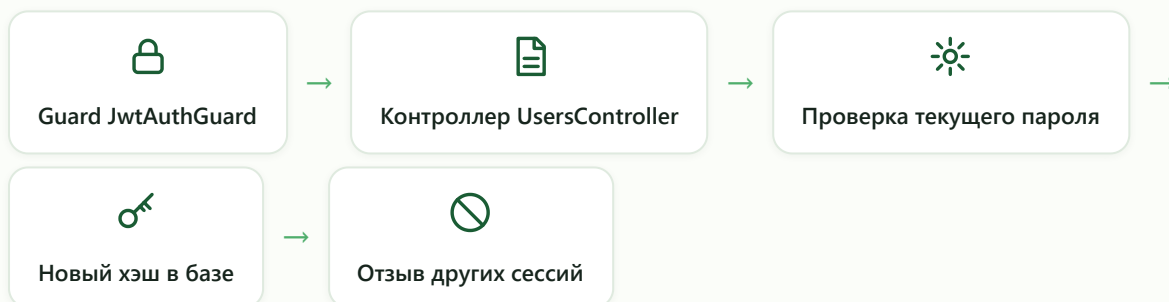
{currentPassword: "33333333", password: "TempDocgen2026!"}
currentPassword: "33333333"
password: "TempDocgen2026!"

← Preview — что сервер ответил

Elements	Console	Sources	Network	Performance	Memory	Application	Security	>>
Name	×	Headers	Payload	Preview	Response	Initiator	Timing	
change-password				▼ {user: {id: "cmu5qvm4e0000jcv6j849ze2i", email: "shipinomaksim@gmail.com", message: "Password changed successfully", success: true}, user: {id: "cmu5qvm4e0000jcv6j849ze2i", email: "shipinomaksim@gmail.com", message: "Password changed successfully", success: true}}				

Что улетело на сервер и что вернулось

Payload — два поля: `currentPassword` (текущий пароль) и `password` (новый). Поле «подтверждение пароля» из формы на сервер **не отправляется** — совпадение двух полей проверяет сам браузер ещё до запроса. **Preview:** `success: true`, сообщение и обновлённый профиль `user` (без пароля и без хэша).



```
// backend/src/users/users.service.ts

async changePassword(userId: string, dto: ChangePasswordDto, currentSessionId?: string) {
  const existingUser = await this.prisma.user.findUnique({ where: { id: userId } });

  const currentPasswordMatches = await bcrypt.compare(
    dto.currentPassword,
    existingUser.passwordHash,
  );

  if (!currentPasswordMatches) {
    throw new UnauthorizedException('Current password is incorrect');
  }

  const passwordHash = await bcrypt.hash(dto.password, 10);
  const user = await this.prisma.user.update({
    where: { id: userId },
    data: { passwordHash },
  });

  // Если пароль скомпрометирован и им уже кто-то пользуется на другом
  // устройстве - эта смена пароля выкидывает его оттуда. Своё же
  // текущее устройство (с которого только что сменили пароль) не трогаем.
  await this.prisma.userSession.updateMany({
    where: { userId, revokedAt: null, id: { not: currentSessionId } },
    data: { revokedAt: new Date() },
  });

  return { user, message: 'Password changed successfully', success: true };
}
```

Зачем спрашивать текущий пароль, если человек и так уже вошёл в аккаунт?

Потому что "вошёл в аккаунт" и "знает пароль" - разные вещи. Например, если кто-то украл активную сессию (токен) через уязвимость в браузере или получил доступ к разблокированному телефону, он сможет открыть настройки, но не будет знать текущий пароль. Требование ввести его ещё раз не даёт злоумышленнику тихо перехватить аккаунт себе, сменив пароль без ведома владельца.

Почему это отдельная форма, а не часть "Редактировать профиль"?

У формы редактирования профиля (имя, e-mail, телефон) на сервере нет ни единого поля для пароля - оно физически отсутствует в структуре допустимых данных этого запроса. Значит, даже теоретически через эту форму пароль сменить нельзя. Единственный путь изменить пароль - через отдельный адрес, который дополнительно требует текущий пароль. Это уменьшает количество мест, где можно случайно (или намеренно) утечь эту возможность.

6

Имя, e-mail, телефон, фото — самая обычная форма «обновить данные».

Имя изменено, кнопка «Save changes» стала активной (до первого изменения она заблокирована).

 Platform

🔍 search store, order, agent...

My account

Your profile, password, two-factor authentication, and where you're signed in



Maksim Shipino (docs demo)

PLATFORM_ADMIN

[Edit profile](#)

Email

shipinomaksim@gmail.com

Phone

3850006879

Password & Security

Manage your password and two-factor authentication

Password

Last changed 2 months ago

Change Password

Two-factor authentication

Extra code required at login, in addition to your password

Enabled

Disable

Role & Access

[Update your profile details](#)

Role

Full access to every module -stores, catalog, payouts, delivery, network, and platform settings.

[View permission matrix →](#)

Active sessions

Devices where you're currently signed in

3 devices signed in

✔ Updated successfully

Уведомление «Updated successfully».

PUT

http://localhost:4000/users/me

200 OK

→ Payload — что браузер отправил на сервер (тело запроса)

ElementsConsoleSourcesNetworkPerformanceMemoryApplicationSecurity>>

NameXHeadersPayloadPreviewResponseInitiatorTiming

me

▼ Request payloadView source

▼ {fullName: "Maksim Shipino (docs demo)", email: "shipinomaksim@gmail.com", email: "shipinomaksim@gmail.com", fullName: "Maksim Shipino (docs demo)", phone: "3850006879", profileImage: "https://github.com/shadcn.png"}

← Preview — что сервер ответил

ElementsConsoleSourcesNetworkPerformanceMemoryApplicationSecurity>>

NameXHeadersPayloadPreviewResponseInitiatorTiming

me

▼ {user: {id: "cmu5qvm4e0000jcv6j849ze2i", email: "shipinomaksim@gmail.com", message: "Updated successfully", success: true}, user: {id: "cmu5qvm4e0000jcv6j849ze2i", email: "shipinomaksim@gmail.com"}}

Что улетело на сервер и что вернулось

Payload — ровно четыре поля: `fullName`, `email`, `phone` и `profileImage`. Поля «пароль» здесь нет и быть не может (см. предыдущий раздел). **Preview**: `success: true`, `message: "Updated successfully"` и свежий профиль `user`.

```
// backend/src/users/users.service.ts
async update(userId: string, dto: UpdateUserDto): Promise<UpdateUserResponse> {
  try {
    const user = await this.prisma.user.update({
      where: { id: userId },
      data: { ...dto }, // только fullName, email, phone, profileImage
      omit: { passwordHash: true, twoFactorSecret: true },
    });
    return { user, message: 'Updated successfully', success: true };
  } catch (error) {
    // Кто-то уже использует этот e-mail - понятная ошибка вместо "500".
    if (error instanceof Prisma.PrismaClientKnownRequestError && error.code === 'P2002') {
      throw new ConflictException('Email is already in use');
    }
    throw error;
  }
}
```

Почему кнопка «Save changes» была неактивна до правки?

Фронтенд сравнивает текущие значения полей с исходными и блокирует сохранение, пока нет ни одного реального изменения - это просто избавляет от лишнего запроса на сервер, когда сохранять на самом деле нечего.

А что если ввести поле, которого нет в форме?

Не получится даже искусственно - на входе в сервер стоит общий фильтр (ValidationPipe с whitelist), который вообще отбрасывает любые поля, не описанные в разрешённом списке для этого запроса, и с настройкой forbidNonWhitelisted вернёт ошибку 400, если такое поле обнаружено.

7 Что защищает аккаунт: итоги

Коротко, без кода - что происходит "под капотом" всегда, вне зависимости от конкретного экрана.

Защита	Что это значит простыми словами
Пароли и коды не хранятся в открытом виде	В базе лежит только необратимый «отпечаток» (хэш). Даже у разработчиков нет способа посмотреть исходный пароль или код.
Одинаковый ответ на неверный email и неверный пароль	Сообщение об ошибке при входе не выдаёт, зарегистрирован ли вообще такой email в системе.
Ограничение попыток ввода кода	После 5 неверных попыток код перестаёт приниматься - перебором его не подобрать.
Ограничение частоты запросов (Rate limiting)	Чувствительные адреса (вход, коды 2FA) не дадут постучаться чаще нескольких раз в минуту с одного места.
Разные секреты для разных токенов	Ключ доступа, ключ обновления сессии и ключ для кода 2FA подписаны разными секретами - утечка одного не подделывает остальные.
Сессии можно отозвать	Даже если токен технически ещё не истёк, запись о сессии в базе можно пометить «отозвана» - и доступ пропадёт мгновенно (например, при выходе или смене пароля).
Смена пароля требует текущий пароль	Захват активной сессии сам по себе не даёт возможности тихо увести аккаунт себе.
Строгая проверка входных данных	Любое поле, не предусмотренное конкретной формой, сервер отклоняет ещё до того, как оно попадёт в бизнес-логику.

Документ сгенерирован автоматически на основе реального прогона приложения (Playwright) поверх фактического кода бэкенда и фронтенда платформы. Дата: 19 сентября 2026 г